



Saad Dahlab University - Blida1

Faculty of Sciences

Department of Computer Science

Computer Science Engineering

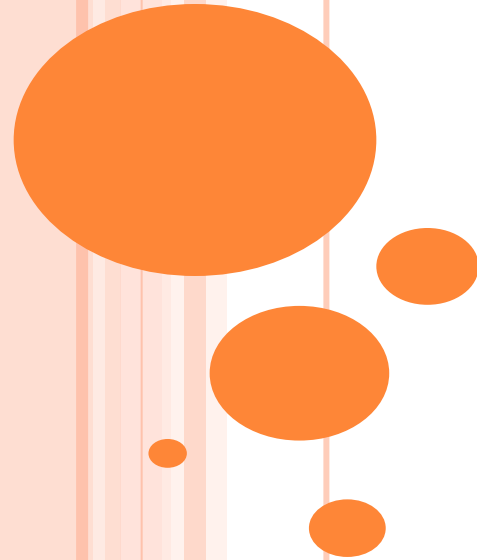
Semester 1 (1st year)

Algorithmic and Data Structures 1



ANNEX: BASIC ELEMENTS OF C LANGUAGE

2025-2026



CONTENT

- ❑ Presentation
- ❑ C Programming
- ❑ C Language Formalism

PRESENTATION

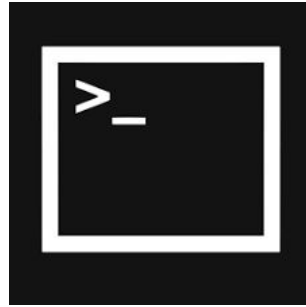
- ❖ Among all existing programming languages, the C language is certainly the most famous. It has often, in the past, been referred to as a universal language in the sense that almost any computer system can be developed from this language. For example, most current operating systems are programmed in C (at least their kernel).
- ❖ It is a low level language (close to the machine) in the sense that it allows access to data (bits, bytes, addresses) that machines manipulate and which are not often available from high level languages such as Java, C#, Python, etc.
- ❖ Compared to other languages (notably Java), the C language makes it possible to develop more compact and much faster applications, but in much more time.
- ❖ It is very important to master the C language in order to better understand the languages of the highest level.

WHY LEARN?

- ❖ **Its popularity:** It has a very large community and many tutorials and documentation. In addition, there are many programs and libraries developed in C.
- ❖ **Its speed:** making it a language of choice for any program where response time (execution) is crucial.
- ❖ **Its lightness:** which makes it useful for embedded programs where the available memory is low.
- ❖ **Its portability:** a program developed in C theoretically works on any platform (Windows, Linus, Unix, Mac, etc.)

REQUIRED TOOLS

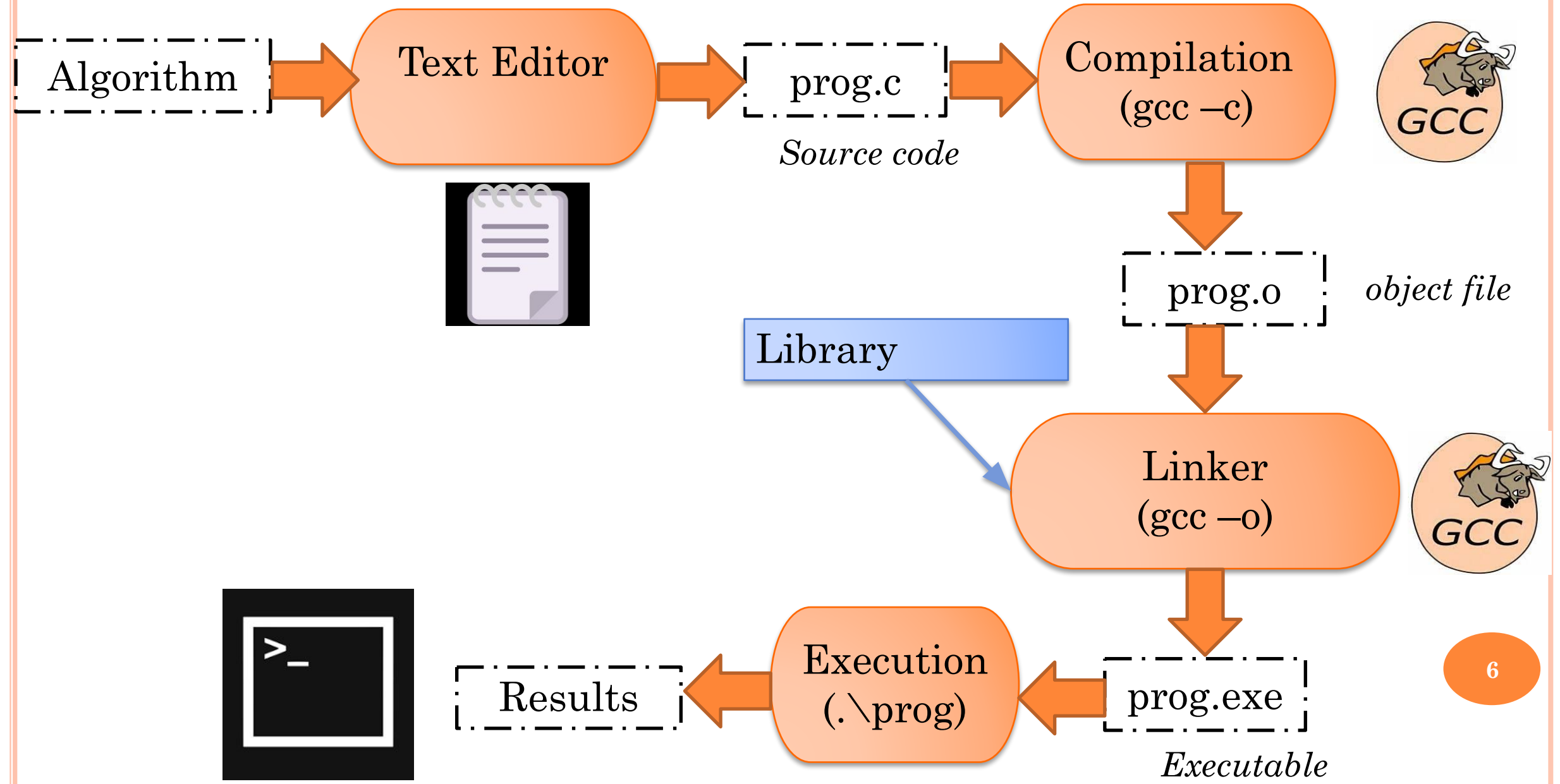
- ❖ Programmers need two tools to program in C:
a text editor and a compiler for the C language.



- ❖ At first (PW1 and PW2), we will use these tools separately in order to understand the process of compilation and execution via the command prompt.
- ❖ But later (PW3 and PW4), we will use a development environment (IDE, Integrated development environment) that integrates these two tools and others (debugger, etc.) as Code::Blocks, Visual C++, etc.



PROGRAMMING



STANDARDS

- ❖ In 1973, the C language was officially announced by Dennis M. Ritchie based on the BCPL and B languages.
- ❖ In 1989, the ANSI (American National Standards Institute) committee standardized the language by creating the C-ANSI standard (also commonly referred to as C89).
- ❖ In 1990, the C language was standardized by ISO (International Standardization Organization), then referred to as C90.
- ❖ In 1999, the ISO committee published a second version of its standard: we then speak of C99 which incorporates many points (Boolean type, comment on a single line, no more obligation to declare the variables at the head of the block, etc.). **In this course, we will use this standard.**
- ❖ Other new versions of the standard have been proposed by ISO: C11 in 2011 and C18 in 2018 which add some new features (threads, bug fix, etc.).

C LANGUAGE FORMALISM

- ❖ C language identifiers
- ❖ They make it possible to give a name to an entity of the program (variable, function, or type)
- ❖ An identifier begins with an alphabetic letter, whether or not followed by alphabetic letters, numbers, or "_".
- ❖ Some rules for choosing identifiers:
 - They must not be chosen from the keywords: *char, double, float, int, long, short, signed, unsigned, enum, const, break, case, default, for, while, do, if, else, switch, return, etc.*
 - They must respect the case (upper or lower case).
 - The compiler may truncate identifiers to a certain length (usually longer than 31 characters).

C LANGUAGE FORMALISM

❖ Structure of a program

```
ALGORITHM <identifier>  
<declarations>  
BEGIN  
< Body: instructions >  
END
```

```
#include <library>  
int main ()  
{  
  <declarations>;  
  < Body: instructions >;  
  return 0;  
}
```

- The declaration part can be in the main function, as it can be before the main function.
- The main function is the entry point of the program: the first one that will be executed.
- Each BEGIN corresponds to an opening brace {
- Each END corresponds to a closing brace }
- Each statement ends with a semicolon ;

C LANGUAGE FORMALISM

❖ Declaration of variables

VAR <identifier> : **TYPE**

TYPE <identifier> ;

❖ Declaration of constants

CONST <identifier> = value

CONST **TYPE** <identifier> = value;

❖ Example:

CONST PI = 3.14

VAR

X: INTEGER

Y: REAL

B: BOOLEAN

C: CHARACTER

CONST **float** PI = 3.14

int X;

float Y;

bool B;

char C;

C LANGUAGE FORMALISM

❖ Standard Type: Integer

Data type	Algorithms	Meaning	N Oct	Range of values
short [int]	integer <i>[int]</i> <i>OPTIONAL</i>	Short integer	2	-32 768 à 32 767
unsigned short [int]		Unsigned integer	2	0 à 65 535
int		Integer	2 4	-32 768 à 32 767 -2 147 483 648 à 2 147 483 647
unsigned [int]		Unsigned integer	2 4	0 à 65 535 0 à 4 294 967 295
long [int]		Long integer	4	-2 147 483 648 à 2 147 483 647
unsigned long [int]		Unsigned Long	4	0 à 4 294 967 295
long long [int]		Double long integer	8	-9 223 372 036 854 775 807 à 9 223 372 036 854 775 807
unsigned long long [int]		Signed double long integer	8	0 à 18 446 744 073 709 551 615

C LANGUAGE FORMALISM

❖ Standard types: real and character

Data type	Algorithms	Meaning	N Oct	Range of values
float	real	Floating or real	4	$3,4 \times 10^{-38}$ à $3,4 \times 10^{38}$ env.
double		Double Float	8	$1,7 \times 10^{-308}$ à $1,7 \times 10^{308}$ env.
long double		Float Double Long	10	$3,4 \times 10^{-4932}$ à $3,4 \times 10^{4932}$ env.
char	char	Character	1	- 128 à 127
unsigned char		Unsigned character	1	0 à 255

C LANGUAGE FORMALISM

- ❖ Standard Type: Boolean
- ❖ There are two ways to declare a variable “b” of the Boolean type in the C99 standard:
 - Declare this variable as an integer (..... int) then test it to find out its logical value: any integer value other than 0 will be considered true and the value 0 will be considered false.
 - Use the header <stdbool.h> that defines three macros, **bool**, **true** and **false** to improve program readability.
 - Declaration: bool b;
 - Logical test: (b == true) or (b == false)

C LANGUAGE FORMALISM

❖ Non-standard types

Type	algorithmic language	C language
Enumerated	<u>TYPE</u> <Id> = (a ₁ , a ₂ , ..., a _n)	enum <Id> {a ₁ , a ₂ , ..., a _n };
Interval	<u>TYPE</u> <Id> = C ₁ .. C _n	It does not exist, just use an integer (int) and a condition that verifies that we are in the interval

C LANGUAGE FORMALISM

❖ Assignment

$\langle \text{Id} \rangle \square \langle \text{valeur} \rangle \text{ ou } \langle \text{Id} \rangle \square \langle \text{identificateur} \rangle \text{ ou } \langle \text{Id} \rangle \square \langle \text{expression} \rangle$

$\langle \text{Id} \rangle = \langle \text{valeur} \rangle \text{ ou } \langle \text{Id} \rangle = \langle \text{identificateur} \rangle \text{ ou } \langle \text{Id} \rangle = \langle \text{expression} \rangle$

❖ Operators

Language/Operators	Algorithmic	C language	Comments
Arithmetic	+, -, *, /, mod, div	+, -, *, /, %	div $\equiv$ integer /integer
Logic	NOT, AND, OR,	!, &&, ,	
Relational	<, <=, >, >=, =, $\neq$,	==, !=, <, <=, >, >=	Equality $\equiv$ double =

C LANGUAGE FORMALISM

❖ Assignment Operators

- Note that in the C language, the assignment is an operator (not an instruction) evaluated from right to left. In addition, there is a set of other operators that are combinations between an arithmetic operator, and the assignment operator.

Operator	Example	equivalent to
+=	<code>a += 10</code>	<code>a = a + 10</code>
-=	<code>a -= 10;</code>	<code>a = a - 10;</code>
*=	<code>a *= 2;</code>	<code>a = a * 2;</code>
/=	<code>a /= 2;</code>	<code>a = a / 2;</code>
%=	<code>a %= 2;</code>	<code>a = a % 2;</code>

C LANGUAGE FORMALISM

- ❖ **The increment operator (++) and decrement operator (--)** are unary operators that can act on an integer (including character) or real variable. The variable is in fact increased by 1 (incrementation) or decreased (decrementation) by 1, while the value returned is either the old value of the variable (post-incrementation or post-decrementation) or the new one (pre-incrementation and pre-decrementation).
- ▣ **Example:** Give result of variables to each statement

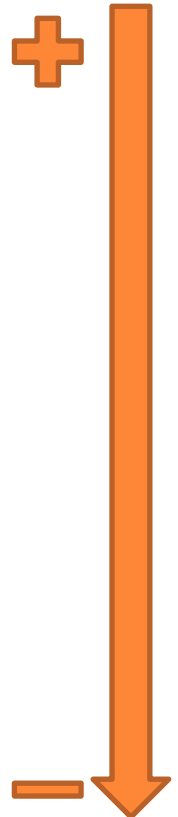
```
int i = 4;  
int u = i++;  
int v = i--;  
int w = --u;  
int t = ++v;
```

Results

```
i = 4;  
u = 4; i = 5;  
v = 5; i = 4;  
u = 3; w = 3;  
v = 6; t = 6;
```

C LANGUAGE FORMALISM

- ❖ **Operator priority:** This table shows the operator precedence table: this is a ranking of the operators of the C language from the highest priority (at the top of the table) to the lowest priority (at the bottom).



Symbol	Operators	Sense of evaluation
Bracket	<code>()</code> , <code>++</code> , <code>--</code> (post)	Left (L) □ Right (R)
unary operators	<code>-</code> , <code>!</code> , <code>++</code> , <code>--</code> (pre)	R □ L
Arithmetic operator	<code>*</code> / <code>%</code>	L □ R
Arithmetic operator	<code>+</code> -	L □ R
Comparison Operators	<code><</code> <code><=</code> <code>>=</code> <code>></code>	L □ R
Comparison Operators	<code>==</code> <code>!=</code>	L □ R
Logical AND	<code>&&</code>	L □ R
Logical OR	<code> </code>	L □ R
Conditional operator	<code>?:</code>	R □ L
Assignment Operators	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	R □ L

C LANGUAGE FORMALISM

❖ Alternative Actions

Algorithmic Language	C Language
<u>if</u> (condition) <u>then</u> <u>BEGIN</u> bloc _{if} <u>END</u> _{if}	<u>if</u> (condition) { bloc _{if} ; }
if (condition) then <u>BEGIN</u> bloc _{if} <u>END</u> _{if} else <u>BEGIN</u> bloc _{else} <u>END</u> _{else}	if (condition) { bloc _{if} ; } else { bloc _{else} ; }

C LANGUAGE FORMALISM

- ❖ **The conditional operator** is used to evaluate a value according to a condition.

```
condition ? ifTrueExpression : ifFalseExpression ;
```

- It is the only operator of the C language that works on three operands: the condition, the value to be calculated if the condition is true and the value to be calculated if the condition is false.
- As a result, this operator is broken down into two parts: **the character "?"** separates the condition from the expression to be evaluated if the condition is true and **the character ":"** to separate the expression associated with the true condition and that associated with the false condition.

C LANGUAGE FORMALISM

- ❖ Alternative Actions: At the first constant whose value is equal to that of expression, the corresponding statement(s) (of one or more blocks) are executed until the first break statement is encountered; indeed, the encounter of a break statement terminates the execution of the switch statement.

Algorithmic Language	C Language
<pre>switch (expression) BEGIN case c_1 : begin bloc₁ end case c_2 : begin bloc₂ end . . case c_n : begin bloc_n end [default: begin bloc_{default} end] END switch</pre>	<pre>switch (expression) { case c_1 : bloc₁ ; [break;] case c_2 : bloc₂ ; [break;] . . case c_n : bloc_n ; [break;] [default: bloc_{default} ;] }</pre>

C LANGUAGE FORMALISM

❖ Iterative actions

Algorithmic Language	C Language
for $ct \square iv$ to fv [step p] do begin bloc end _{for}	for ($ct=iv$; $ct \leq fv$; $ct=ct+p$) { bloc; }
while ($condition$) do begin bloc end _{while}	while ($condition$) { bloc; }
do begin bloc end while ($condition$)	do { bloc; } while ($condition$)

C LANGUAGE FORMALISM

❖ Iterative actions: FOR

- The **for** loop is broken down into three separately optional parts (or three clauses).
 - ▢ **Initialization** allows you to define loop start conditions,
 - ▢ **Condition** is a Boolean expression that must be true for the continuation of the loop, i.e. a new execution of < instruction block > ,
 - ▢ **Iteration** performs a state change of the variable (s) used in the loop iteration control.
- The **Initialization** and **Iteration** clauses can be composed of multiple statements separated by a comma

```
for (<Initialization>; <Condition>; <Iteration>)  
{  
    <instruction block>;  
}
```

```
for (int j=0, int i = j+5 ; (j<10) || (i<100); j++, i* = 2)  
    printf("j = %d , i = %d ", j, i) ;
```

C LANGUAGE FORMALISM

- ❖ Reading - Writing Functions
 - ❖ The header `<stdio.h>` (short for **standard in-out**) provides three streams of data that can be used directly:
 - `stdin`, the standard input that sends the data from the keyboard to the program.
 - `stdout`, the standard output that sends the data that the program generates to the screen.
 - `stderr`, the standard output of the errors that will be displayed on the screen.
 - ❖ This library must be included in the C program (before the main function) in order to use the reading (`scanf`) and writing (`printf`) functions.

C LANGUAGE FORMALISM

❖ Reading

- The "scanf" function makes it possible to read values on the standard input unit (stdin), according to a format specified as an argument and write them into memory cases whose addresses are provided as arguments.

```
scanf (format, arg1,arg2,...,argn);
```

- ❖ Often, the arguments arg_i have memory addresses where variables are stored and are therefore of the form &var_i where var_i is the identifier of a variable and & is the addressing operator:

```
scan ([f],var1, var2, ... , varn)
```

```
scanf (format, & var1, & var2,..., & varn)
```

C LANGUAGE FORMALISM

❖ Reading

```
scanf (format, & var1, & var2,..., & varn )
```

- The "format" string contains the format specifications of the variables (var₁, var₂,..., var_n) to be retrieved from the input stream.
- Introduced by the % sign, these specifications are:

code	Type of variable/expression to read/write
%	Read a percentage % (use %%)
d ou o ou x	signed integer expressed in octal, decimal, or hexadecimal base
h, l	signed short or long integer expressed in decimal base
u	Unsigned integer
e ou f ou g	number with decimal part in decimal point notation, in exponential notation or the shortest (writing)
c	character

- If the value of a variable is not of the type indicated by the format, it will be converted.

C LANGUAGE FORMALISM

Declaration	reading	writing	external format
int i;	scanf("%d",&i);	printf("%d",i);	decimal
int i;	scanf("%o",&i);	printf("%o",i);	octal
int i;	scanf("%x",&i);	printf("%x",i);	hexadecimal
unsigned int i;	scanf("%u",&i);	printf("%u",i);	decimal
short j;	scanf("%hd",&j);	printf("%d",j);	decimal
short j;	scanf("%ho",&j);	printf("%o",j);	octal
short j;	scanf("%hx",&j);	printf("%x",j);	hexadecimal
unsigned short j;	scanf("%hu",&j);	printf("%u",j);	decimal
long k;	scanf("%ld",&k);	printf("%d",k);	decimal
long k;	scanf("%lo",&k);	printf("%o",k);	octal
long k;	scanf("%lx",&k);	printf("%x",k);	hexadecimal
unsigned long k;	scanf("%lu",&k);	printf("%u",k);	decimal

Integers can be expressed in three different numerical bases, by:

- decimal (the most classic use)
- octal (the number is simply prefixed with a character 0)
- hexadecimal (prefixed with both 0x characters).

C LANGUAGE FORMALISM

❖ Example

Declaration	reading	writing	external format
float l;	scanf("%f",&l);	printf("%f",l);	Decimal point
float l;	scanf("%e",&l);	printf("%e",l);	Exponential
float l;		printf("%g",l);	SHORTEST
double m;	scanf("%lf",&m);	printf("%f",m);	Decimal point
double m;	scanf("%le",&m);	printf("%e",m);	Exponential
double m;		printf("%g",m);	SHORTEST
long double n;	scanf("%Lf",&n);	printf("%Lf",n);	Decimal point
long double n;	scanf("%Le",&n);	printf("%Le",n);	Exponential
long double n;		printf("%Lg",n);	SHORTEST
char o;	scanf("%c",&o);	printf("%c",o);	character

The reals are represented in floating point which can be expressed in notation:

- decimal (the most conventional use), e.g:
double nd = 3.14159254;
- Exponential (*sign* 0, *mantissa* B^{exponent}),
e.g double ne = 3.2e-10;

C LANGUAGE FORMALISM

❖ Writing

- The "fprintf" function allows values to be written to the standard output unit (stdout or screen), according to the "format" specified in the argument.

print ([f], exp₁, exp₂, ... , exp_n)

printf (format, exp₁, exp₂, ... , exp_n)

- As well as reading, the "format" parameter contains the characters to be displayed and the writing format specifications corresponding to the parameters exp1, exp2 , ..., expn .
- Strings of type %zz will be replaced on printing by the values of expressions exp1, exp2, ... expn.
- If the value of an expression is not of the type indicated by the format, it will be converted.

C LANGUAGE FORMALISM

❖ Standard mathematical functions

- The `<stdlib.h>` and `<math.h>` libraries provide the main mathematical functions:

Library	Function	Description
<stdlib.h>	abs	Absolute value of integers (int)
	labs	Absolute value of long integers
	div	Obtaining quotient and remainder
<math.h>	fabs	Absolute value of floats
	pow	Power
	sqrt	Square root
	exp	Exponential
	log	Logarithm
	cos, acos	cosine, arc cosine
	sin, asin	sine, arc sine
	tan, atan	tangent, arc tangent

C LANGUAGE FORMALISM

- ❖ Comments:
- ❖ To comment on code written in program C, you can use:
 - ❑ **The /* and */ symbols** surrounding the comment. The latter can extend over several lines, *e.g /* This is a comment on several possible lines */*
 - ❑ **The // symbol** (two slashes) preceding the comment, limited to the rest of the line, *e,g: // This is a one-line comment*
- ❖ Comments can be placed anywhere in the program without cutting a word of the program in half.
- ❖ They are ignored when running the program.
- ❖ Their use, although optional, is fundamental. Indeed, they allow an easier understanding of a code left behind for weeks and are useful when maintaining programs

C LANGUAGE FORMALISM

- ❖ **Example:** translate the following algorithm into C program

```
ALGORITHM abs_parity
VAR x : INTEGER
BEGIN
  PRINT (" give en integer x = ")
  SCAN (X)
  IF (x >= 0) THEN    PRINT (" The absolute value of is : ", x )
  ELSE    PRINT (" The absolute value of is : ", (-1) * x)
  IF (x MOD 2 = 0) THEN PRINT ("x est even ")
  ELSE    PRINT ("x est odd ")
END
```

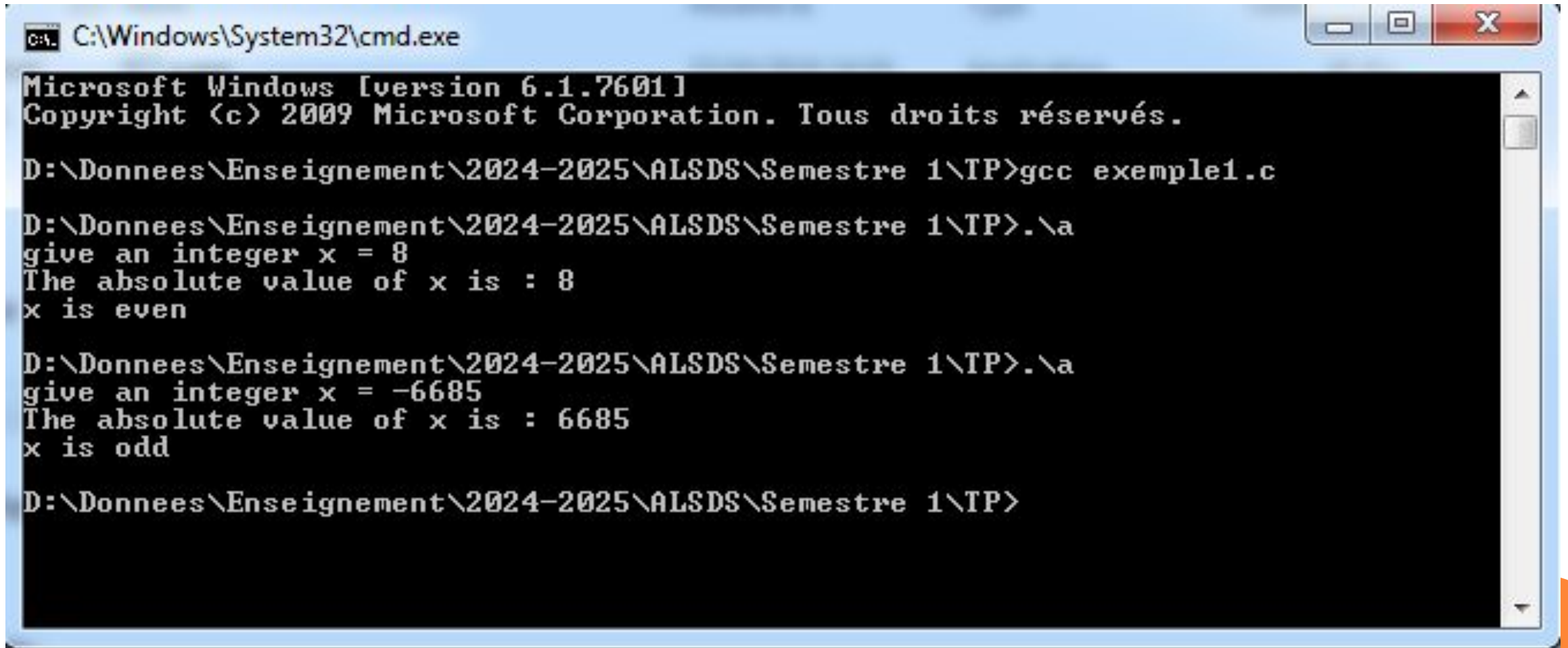
C LANGUAGE FORMALISM

- ❖ **Example:** translate the following algorithm into C program

```
exemple1.c
1  #include <stdio.h>
2  #include <math.h>
3  int main ()
4  {
5      int x;
6      printf("give an integer x = ");
7      scanf ("%d", &x);
8      printf("The absolute value of x is : %d \n", abs(x));
9      if (x % 2 == 0)
10         printf("x is even \n");
11         else
12             printf ("x is odd \n");
13     return 0;
14 }
```

C LANGUAGE FORMALISM

- ❖ **Example:** translate the following algorithm into C program



```
C:\Windows\System32\cmd.exe
Microsoft Windows [version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Tous droits réservés.

D:\Donnees\Enseignement\2024-2025\ALSDS\Semestre 1\TP>gcc exemple1.c

D:\Donnees\Enseignement\2024-2025\ALSDS\Semestre 1\TP>.\a
give an integer x = 8
The absolute value of x is : 8
x is even

D:\Donnees\Enseignement\2024-2025\ALSDS\Semestre 1\TP>.\a
give an integer x = -6685
The absolute value of x is : 6685
x is odd

D:\Donnees\Enseignement\2024-2025\ALSDS\Semestre 1\TP>
```

COURSE SOURCES

- S. AROUSSI (2017), Cours Algorithmique & Structures de Données (ASD), 2nd year licence, Computer Science Department, Saad Dahlab University, Blida.
- Ressources Pédagogiques des programmeurs, Available at <https://koor.fr/C/Index.wp>. Accessed October 2023.